1 Лекция 13. Динамика по подмножествам: Set Cover и Steiner Tree

1.1 Перебираем части маленького множества

Динамическое программирование по подмножествам (subset DP) полезно, когда есть маленькая вселенная обязательств: элементы, которые надо покрыть, или терминалы, которые надо соединить. Сам граф или список доступных множеств может быть большим. Состояний по маленькой вселенной 2^p; это допустимо для FPT по p.

Первая задача — **Set Cover**: выбираем несколько заданных множеств, чтобы их объединение содержало всю вселенную. В Hitting Set, напротив, выбираем элементы, пересекающие каждое требование. Эти постановки связаны, но объект выбора различен. Если $U = \{a, b, c\}$, а доступные множества — $\{a, b\}$ и $\{b, c\}$, минимальное Set Cover состоит из обоих множеств.

Вторая задача — **Steiner Tree**: в графе с весами рёбер соединяем заданные терминалы сетью минимального веса; нетерминальные вершины использовать разрешено. Если три терминала — листья звезды с единичными рёбрами, оптимальная сеть использует нетерминальный центр и стоит 3. При замене задачи поиском дерева только на терминалах можно потерять совместное использование рёбер.

Ниже обе динамики выводятся из смысла состояния. В Set Cover решение разбивается на «берём последнее доступное множество / не берём». В Steiner Tree выбираются точка соединения и разбиение терминалов между двумя частями. Важно доказать как достижимость кандидатов перехода, так и то, что хотя бы один переход описывает оптимум.

1.2 Set Cover: параметр может быть размером вселенной

Дана вселенная U, $|U| = p$, и семейство $F_1, \ldots, F_m$. Ищем минимальное число множеств, покрывающих U. Параметр этой лекции — p, а не число выбранных множеств. Для $X \subseteq U$ и $0 \le i \le m$ пусть $D_i(X)$ — минимум числа множеств среди первых i, покрывающих X; разрешено покрывать и элементы вне X.

База: $D_0(\varnothing) = 0$, остальные значения бесконечны. Переход:

$$D_i(X) = \min\{D_{i-1}(X),\ 1 + D_{i-1}(X \setminus F_i)\}.$$

В оптимальном решении либо нет F_i, либо после его удаления остаётся покрыть $X \setminus F_i$. Обратно, оба кандидата справа дают покрытие X. Это две стороны корректности. Ответ $D_m(U)$, время $O(2^p mp)$ при простых битовых операциях, память $O(2^p)$ значений при хранении двух слоёв. Для восстановления удобно сохранить решения переходов всех слоёв или повторно вычислить нужные префиксы.

2^p-алгоритм не показывает FPT по бюджету числа множеств k. Если p огромен относительно k, экспонента остаётся огромной. В задачах лекции 14 увидим, почему такое различие существенно.

1.3 Один переход Set Cover численно

Пусть $U = \{1, 2, 3\}$, $F_1 = \{1, 2\}$, $F_2 = \{2, 3\}$. После первого слоя $D_1(\{1, 2\}) = 1$, $D_1(\{1\}) = 1$, но $D_1(U) = +\infty$. На втором:

$$D_2(U) = \min\{D_1(U), 1 + D_1(U \setminus F_2)\} = \min\{+\infty, 1 + D_1(\{1\})\} = 2.$$

Для $X = \{2\}$ уже $D_1(X) = 1$, хотя выбранное множество также покрывает 1. Состояние требует покрыть X, а не получить объединение ровно X; из этого различия следует правильность операции $X \setminus F_i$.

1.4 Steiner Tree

Дан неориентированный граф G, множество терминалов K, $|K| = p$, положительные целые веса рёбер в двоичной записи. Нужно связное поддерево минимального веса, содержащее K. Лишние вершины разрешены и называются Steiner vertices. При $p \le 1$ стоимость 0; если терминалы лежат в разных компонентах — решения нет. Далее оставляем компоненту, содержащую все терминалы.

Для удобного доказательства сделаем каждый терминал листом. Для каждого исходного $t \in K$ добавим новый лист t' с ребром веса 1, сделаем t' терминалом вместо t. Для $p \ge 2$ любое решение обязано содержать все новые листовые рёбра. Оптимум увеличился ровно на p; после вычисления эти рёбра удалим. Число терминалов не изменилось. Ниже G, K обозначают уже преобразованный вход.

1.5 Состояние и рекуррентная формула

Для непустого $S \subseteq K$ и нетерминальной вершины v определим $T(S, v)$ — минимум веса дерева, содержащего $S \cup \{v\}$. Не запрещаем присутствие других терминалов, хотя лишние положительные листовые рёбра в оптимуме не нужны.

База $T(\{s\}, v) = dist(s, v)$. Для $|S| \ge 2$:

$$T(S, v) = \min_{u \notin K} \min_{\varnothing \neq A \subsetneq S} \big(T(A, u) + T(S \setminus A, u) + dist(u, v)\big).$$

Сначала можно вычислить слияние $M(S, u) = \min_A [T(A, u) + T(S \setminus A, u)]$, затем $T(S, v) = \min_u [M(S, u) + dist(u, v)]$. Расстояния вычисляются заранее.

1.6 Две стороны доказательства

Любые два дерева для $A \cup \{u\}$ и $(S \setminus A) \cup \{u\}$ вместе с путём u–v образуют связный подграф, содержащий $S \cup \{v\}$. Пересечения могут только уменьшить вес объединения. Удаляем циклы и лишние ветви, получая дерево не тяжелее суммы. Поэтому оптимум слева не больше каждого выражения справа.

Для обратного направления возьмём оптимальное дерево, укоренённое в v. Терминалы — листья, и их минимум два. Идём от v до первой вершины u с минимум двумя дочерними ветвями, содержащими терминалы. Лишних нетерминальных листьев в оптимуме нет: их удаление уменьшило бы вес. Поэтому такая u существует. Путь от v до u не содержит терминалов, а u нетерминальна. Одну ветвь выделяем как первую группу терминалов A, остальные как $S \setminus A$.

Полученные два поддерева с корнем u и путь до v рёберно не пересекаются и вместе дают исходное дерево. Каждую часть можно оценить снизу соответствующим оптимумом T или расстоянием. Значит, один кандидат справа не превосходит оптимума слева. Обе оценки дают равенство.

1.7 Почему слияния недостаточно без переноса корня

Представим дерево с корнем v, длинным путём до развилки u и двумя терминальными ветвями от u. Если соединять два частичных решения только в v, общий путь от v до u оплатится дважды. Формула сначала объединяет части у u, а затем один раз добавляет расстояние до v.

На преобразованной звезде с тремя исходными терминальными листьями добавлены ещё три единичных листовых ребра. От центра u до каждого нового терминала расстояние 2. Поэтому $T(\{s\}, u) = 2$, для пары терминалов минимум 4, для тройки 6. Вычитаем три добавленных ребра и получаем исходную стоимость 3. Нижняя оценка 6 в преобразованном графе очевидна: все шесть рёбер единственных путей к терминалам необходимы.

1.8 Время и результат

Заполняем состояния по возрастанию $|S|$. Число пар (S, A) с $A \subseteq S \subseteq K$ равно 3^p: каждый терминал вне S, в A или в $S \setminus A$. Поэтому все слияния занимают $O(3^p n)$ операций, последующие минимизации по u, v — $O(2^p n^2)$, плюс полиномиальный поиск расстояний. Память $O(2^p n + n^2)$; битовая длина сумм весов полиномиальна по входу.

Ответ на преобразованном графе — $\min_{v \notin K} T(K, v)$. Запоминаем минимизирующие u, A и кратчайшие пути, объединяем восстановленные части, удаляем циклы; затем убираем добавленные терминальные листья и вычитаем p. Самопроверка: почему нельзя взять MST только на исходных терминалах? Где положительность весов используется в доказательстве?

1.9 Практика

Условия задач 13.1–13.5. Решения открывайте после своей попытки.

1.10 Источники и дополнительное чтение

Письменный источник: Cygan et al., *Parameterized Algorithms*, авторская копия 30.05.2016: §6.1, с. 130–134, теоремы 6.1 и 6.3. PDF. Номера страниц печатные; номер страницы PDF больше на 16.

Изложение, поясняющие примеры и упражнения подготовлены для этого курса. Ссылки фиксируют происхождение результатов; для чтения конспекта видео не требуется. Необязательные видео собраны в программе.